EE-599: Designing and Building Autonomous AI Agents

University of Southern California

Ming Hsieh Department of Electrical and Computer Engineering

Scaling Agent Systems

Instructor:
Arash Saifhashemi

EE 599 · AGENTIC PROGRAMMING

Scaling Agentic Systems

Session 04 · Part II of II · Multi-agent orchestration, coordination, and evidence

CENTRAL QUESTION

When does dividing work among agents improve the verified result?

How should work, information, and responsibility move across agents?

Lecture map

NAMED PARTS

1. Orchestrator responsibilities
2. Routing, handoff, and delegation
3. Topologies, peer debate, and failure modes
4. Evaluation and research evidence
5. Design selection

STUDENTS WILL BE ABLE TO

1. Distinguish routing, delegation, and handoff.
2. Compare common topologies and the peer-debate process.
3. Diagnose coordination and responsibility failures from traces.
4. Interpret the paper's evidence and choose a design from task structure.

Retained slides preserve their source-deck numbers.

Orchestration coordinates several bounded agents

AGENT

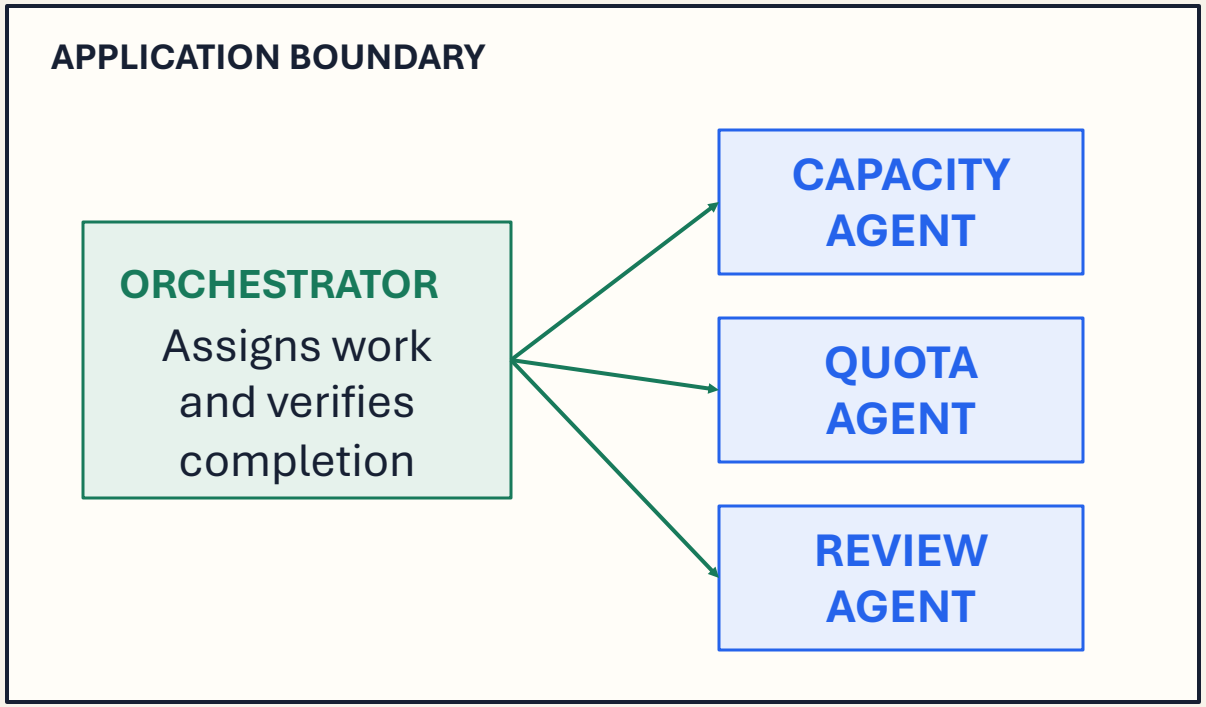
A model-driven participant assigned a bounded role, context, and allowed actions.

MULTI-AGENT SYSTEM

An application that uses more than one agent during one task or interaction.

ORCHESTRATOR

The application role that assigns work, controls information flow, enforces limits, and decides when the system may finish.

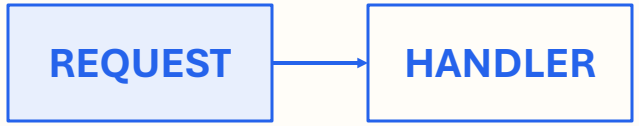


Deck 1 covers action sequencing inside a controlled workflow. This deck covers allocation, communication, and responsibility across agents.

Routing, delegation, and handoff change different responsibilities

ROUTING

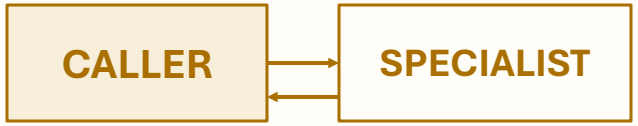
Select the initial handler before work begins.



Selected handler owns the first response.

DELEGATION

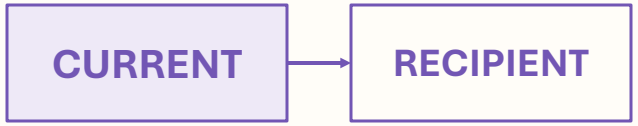
Send a bounded subtask to another agent, receive its result, and keep responsibility with the caller.



Caller owns the next response.

HANDOFF

Transfer the active interaction so the recipient owns later user-facing responses.

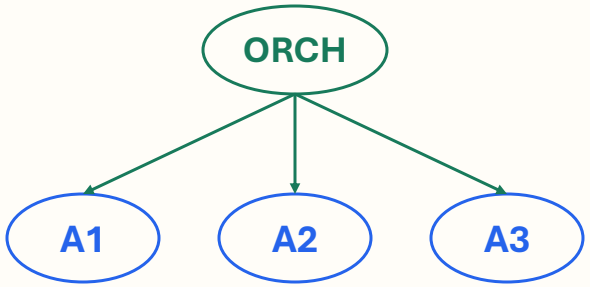


Recipient owns the next response.

Topology determines who communicates and who combines answers

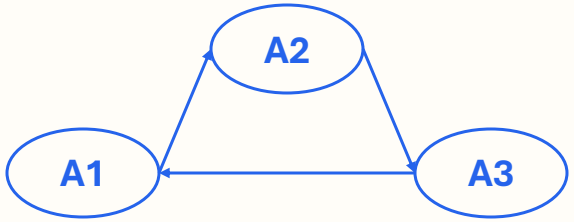
Topology: the allowed communication and control relationships among agents.

STAR COORDINATION



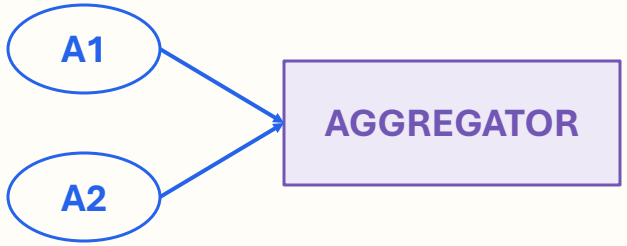
One orchestrator exchanges work and results with bounded agents.

PEER DEBATE



Agents produce answers, exchange bounded critiques, and revise or defend them.

AGGREGATION

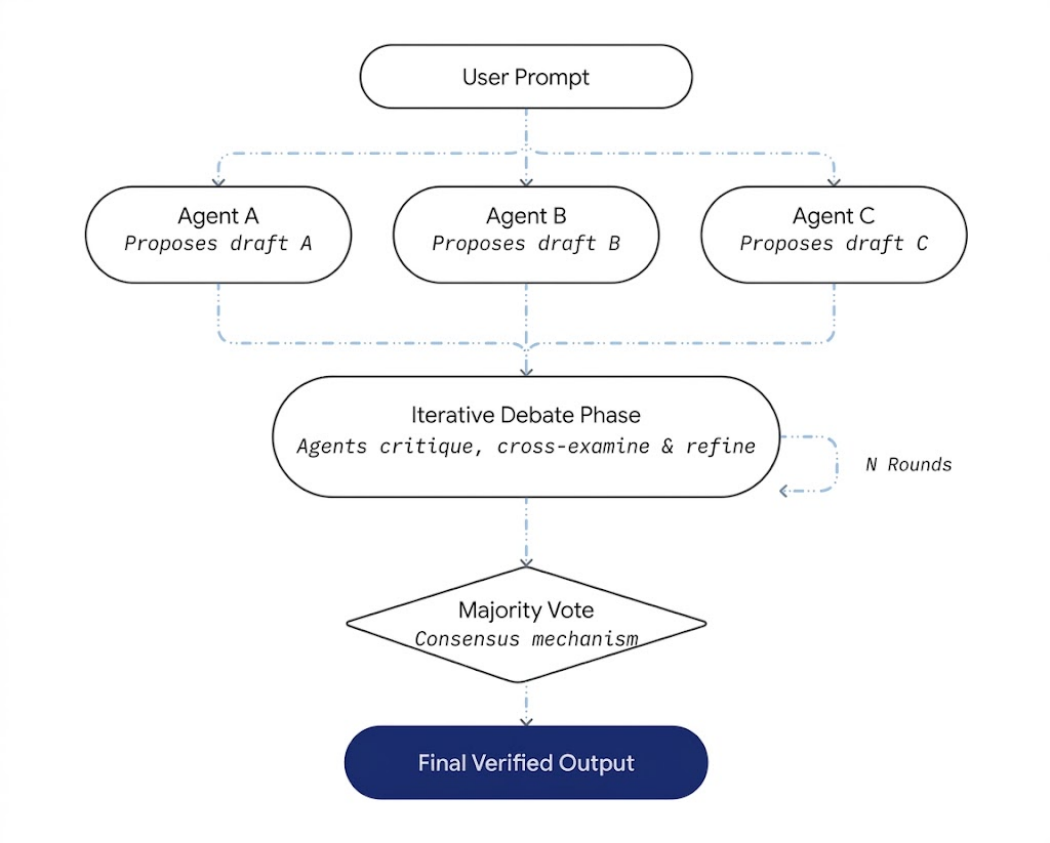


A separate step selects or combines candidate answers before application verification.

SECTION 5

Multi-Agent Orchestration

Application-owned coordination assigns work, controls information flow, and decides when several agents may finish.



DEFINITION

The orchestrator is an application role

Multi-agent orchestration is the application-owned policy that coordinates several agents from task assignment through the final response.

APPLICATION ORCHESTRATOR

May ask models for plans, critiques, or rankings.

Application code keeps authority.

ASSIGN WORK

Create bounded tasks and allowed tool sets.

TRACK PROGRESS

Record results, failures, and pending work.

COMBINE OUTPUTS

Join findings or select one candidate.

CONTROL INFORMATION

Choose which outputs may influence each agent.

ENFORCE LIMITS

Apply round, token, time, and permission limits.

STOP AND RETURN

Run exact checks and produce one terminal response.

An orchestrator can use model assistance without transferring permissions, stopping authority, or ownership of the final response.

Five multi-agent design decisions

One run combines five independent choices. Agent count alone does not determine the architecture.

1	**CONTROL METHOD** How are actions selected?	Fixed workflow, step-by-step loop, or planner-based workflow	**A100 example: fixed workflow**

Each added decision answers a different system question.

Five multi-agent design decisions

One run combines five independent choices. Agent count alone does not determine the architecture.

1	**CONTROL METHOD** How are actions selected?	Fixed workflow, step-by-step loop, or planner-based workflow	**A100 example: fixed workflow**
2	**RESPONSIBILITY** How does responsibility move?	Routing, delegation, handoff, or none	**A100 example: delegation**

> Each added decision answers a different system question.

Five multi-agent design decisions

One run combines five independent choices. Agent count alone does not determine the architecture.

1	**CONTROL METHOD** How are actions selected?	Fixed workflow, step-by-step loop, or planner-based workflow	**A100 example: fixed workflow**
2	**RESPONSIBILITY** How does responsibility move?	Routing, delegation, handoff, or none	**A100 example: delegation**
3	**TOPOLOGY** Whose output may influence whom?	Topology means the allowed information-flow relationships	**A100 example: centralized**

> Each added decision answers a different system question.

Five multi-agent design decisions

One run combines five independent choices. Agent count alone does not determine the architecture.

1	**CONTROL METHOD** How are actions selected?	Fixed workflow, step-by-step loop, or planner-based workflow	**A100 example: fixed workflow**
2	**RESPONSIBILITY** How does responsibility move?	Routing, delegation, handoff, or none	**A100 example: delegation**
3	**TOPOLOGY** Whose output may influence whom?	Topology means the allowed information-flow relationships	**A100 example: centralized**
4	**AGGREGATION RULE** How do outputs become one candidate?	Join, synthesis, vote, consensus, judge, or score	**A100 example: join both findings**

Each added decision answers a different system question.

Five multi-agent design decisions

One run combines five independent choices. Agent count alone does not determine the architecture.

1	**CONTROL METHOD** How are actions selected?	Fixed workflow, step-by-step loop, or planner-based workflow	**A100 example: fixed workflow**
2	**RESPONSIBILITY** How does responsibility move?	Routing, delegation, handoff, or none	**A100 example: delegation**
3	**TOPOLOGY** Whose output may influence whom?	Topology means the allowed information-flow relationships	**A100 example: centralized**
4	**AGGREGATION RULE** How do outputs become a candidate?	Join, synthesis, vote, consensus, judge, or score	**A100 example: join both findings**
5	**VERIFICATION** Who checks the selected candidate?	Application-owned exact acceptance conditions	**A100 example: six exact checks**

A complete design names all five choices and keeps final verification under application control.

SUBSECTION 5A

Routing and handoff between agents

How does work or user-facing responsibility move between agents?

TECHNICAL MAPPING

Agent routing selects the next handler; it does not complete the request.



A telephone switchboard operator connects each caller to the right destination.
Library of Congress, Matson Collection;

Three multi-agent coordination operations

A control method determines how actions are selected. A coordination operation determines how work or responsibility moves between agents.

ROUTING

Switchboard

Select the initial handler before work begins.

The router does not solve the request.

DELEGATION

Manager → analyst

Send a subtask and receive a result.

The caller resumes and remains responsible.

HANDOFF

Doctor → surgeon

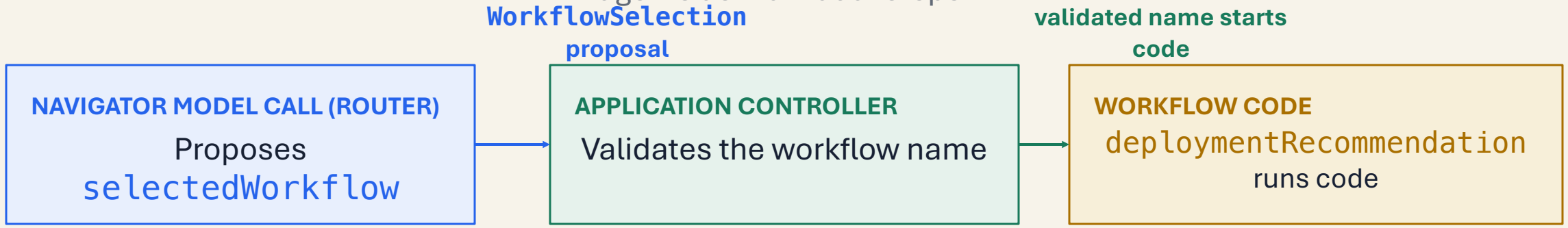
Transfer the active interaction to another agent.

The recipient becomes responsible for later user-facing responses.

> Any coordination operation may appear inside a fixed workflow, step-by-step loop, or planner-based workflow.

Workflow execution

A workflow is not necessarily an agent and is not a prompt. It may invoke models, tools, or specialist agents as individual steps.



COMMON IMPLEMENTATIONS

Python or JavaScript function	State machine or graph
Workflow engine	SDK runner with controlled transitions

> Route selection chooses a workflow; workflow execution defines the ordered steps and failure behavior.

Fixed workflow code defines order and failure behavior

Application code for the fixed workflow

```
def runDeploymentRecommendation(request):
    capacity = checkCapacity(request)
    quota = checkQuota(request)
    candidate = callRecommendationModel(
        request, capacity, quota
    )
    report = verifyRecommendation(
        request, candidate, capacity, quota
    )

    if not report.passed:
        return ExplicitFailure.fromReport(report)
    return VerifiedRecommendation.fromCandidate(
        candidate, report
    )
```

MODEL CALL
Returns a candidate record.

EXACT CHECKS
Checks records, values, citations, and trace.

TERMINAL RESPONSE
Returns checked success or failure.

Workflow selection preserves policy

Navigator proposal

```
{
  "requestId":
    "req-599-04",
  "selectedWorkflow":
    "deploymentRecommendation"
}
```

CONTROLLER CHECK

Look up `selectedWorkflow` in `WORKFLOWS`.

Unknown name → `ExplicitFailure`.

Request policy is unchanged

```
mode = "recommendOnly"
forbiddenActions = [
    "provisionGpu",
]
```

ALLOWED NAME

Start the corresponding application function.

UNKNOWN NAME

Return `ExplicitFailure` before any workflow function starts.

> Selecting a workflow neither executes it nor changes the request's permissions.

Agent handoff transfers an active interaction

DEFINITION

Another agent becomes responsible for continuing the interaction and producing subsequent user-facing responses.

BEFORE

`IntakeAgent` is active and owns the next response

→

APPLICATION CHECK

Validate target, transferred context, and permissions

→

AFTER

`TechnicalSupportAgent` is active and owns later responses

A handoff can occur inside any control method: a predefined workflow step, a validated ReAct-style action, or a validated `PlanStep`.

> Delegation returns a subtask result to the caller. Handoff changes which agent continues the interaction.

`HandoffRequest` is only a proposal

Example: handoff inside a step-by-step agent loop (ReAct-style)

Active agent proposes a `HandoffRequest`

```
{
  "requestId": "support-1042",
  "targetAgent": "technicalSupport",
  "reasonCode": "technicalIssue",
  "contextSummary": "Connection failure.",
  "allowedTools": [
    "readAccount", "runDiagnostics"
  ]
}
```

STATUS: PROPOSED

The controller has not validated the request or activated the target yet.

WHAT APPLICATION CODE CHECKS

- target is allowed
- context is sufficient
- tools are permitted and narrow
- `requestId` matches

`HandoffRequest` is a proposal record, not proof that the transfer occurred.

Handoff phase 1: propose a transfer

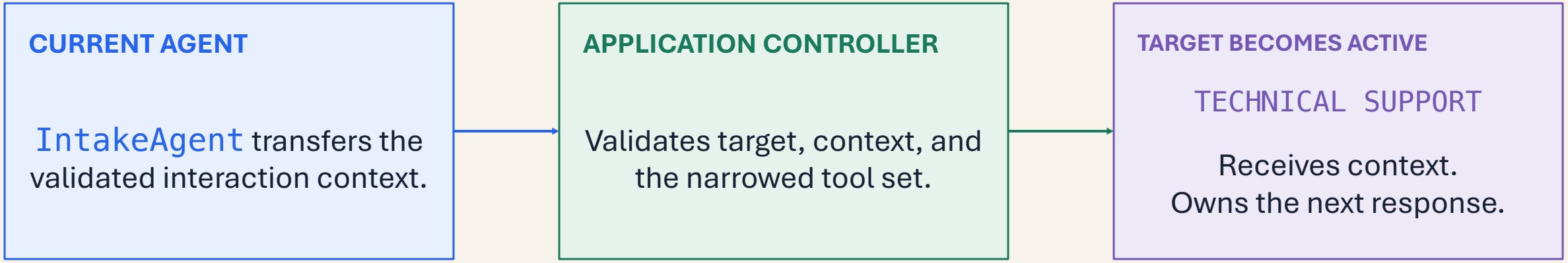
Example: handoff inside a step-by-step agent loop (ReAct-style)



> The current agent remains active until the handoff proposal passes validation.

Handoff phase 2: validate, then activate

Example: handoff inside a step-by-step agent loop (ReAct-style)



> The target becomes active only after application validation passes.

SUBSECTION 5B

Delegation to specialist agents

How can the Application Controller assign a narrow task while retaining responsibility for the user response?



Who remains responsible for the final response?

SCENARIO

The Application Controller sends a `DelegatedTask` to the capacity specialist. The specialist calls `checkCapacity` and returns a `CapacityFinding`. The controller still has the original user request.

A · APPLICATION CONTROLLER

The caller resumes after receiving the finding.

B · CAPACITY SPECIALIST

The specialist produced the latest result.

C · WHICHEVER RETURNS LAST

Responsibility follows the most recent message.

> Vote before the reveal: who owns the next user-facing response?

Delegation returns findings to the caller

Delegation assigns a narrow subtask. It does not transfer the active user-facing interaction.

CAPACITY SPECIALIST TASK

Objective: Is one A100 available?

Context: us-west-2, quantity 1, tonight
Allowed tool: `checkCapacity`
Required output: `CapacityFinding`

QUOTA SPECIALIST TASK

Objective: May EE use one A100?

Context: EE, quantity 1, tonight
Allowed tool: `checkQuota`
Required output: `QuotaFinding`

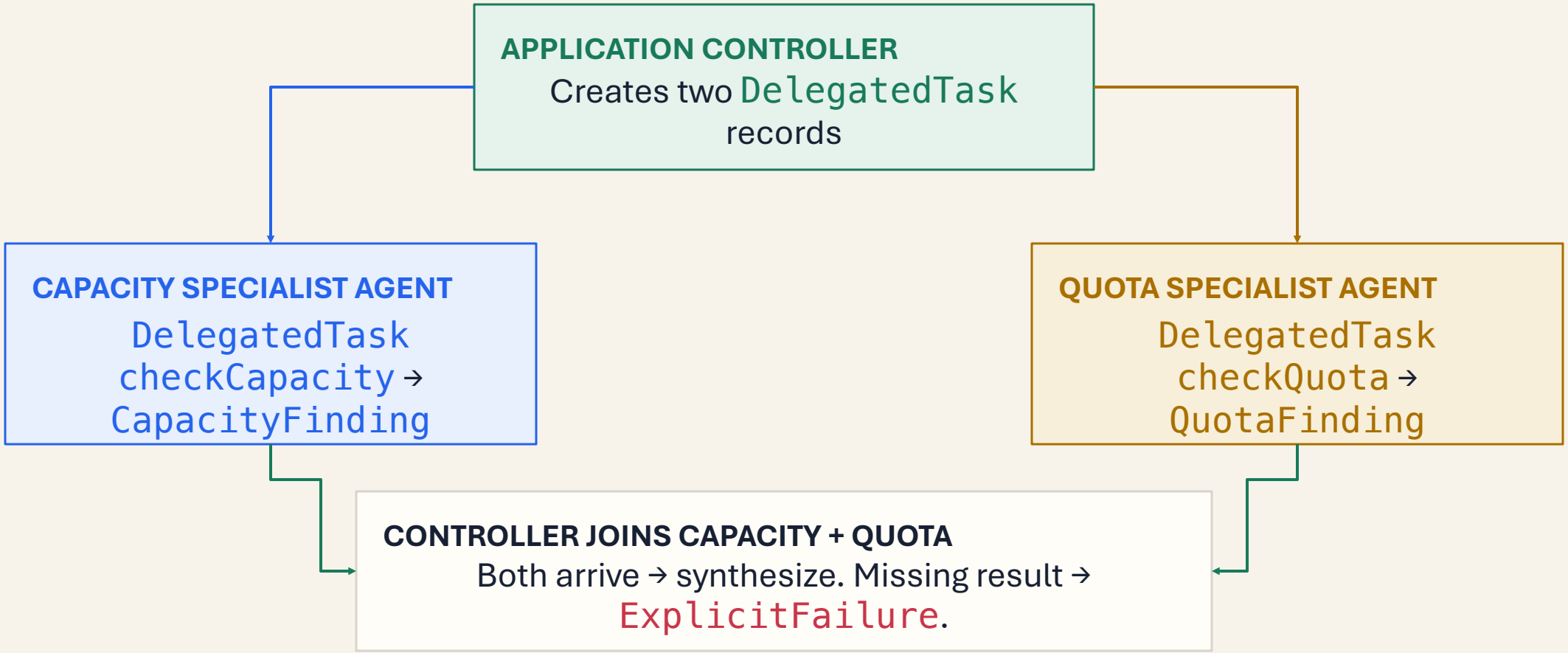
CALLER
`ApplicationController`

FORBIDDEN
`provisionGpu`

ON ERROR
return `ExplicitFailure`

> The controller joins both findings and owns the final response.

Delegation fan-out and join



The controller remains responsible through fan-out, join, synthesis, and the final decision.

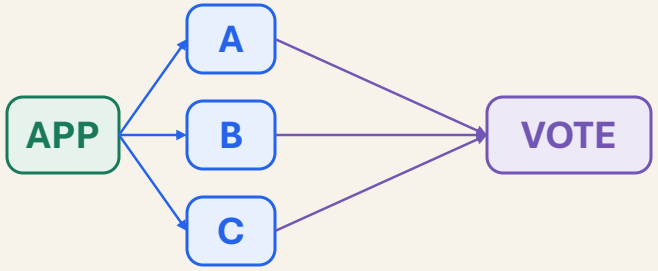
Five common agent-system topologies

A communication topology specifies which participant's output may influence another participant's next action or answer.

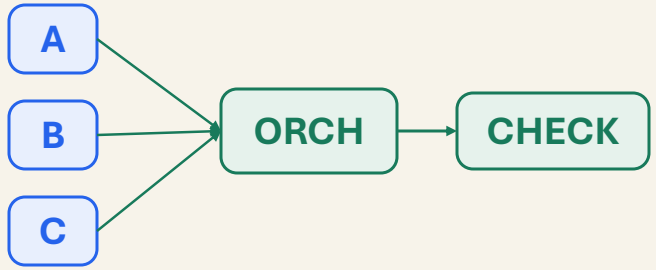
SINGLE-AGENT BASELINE



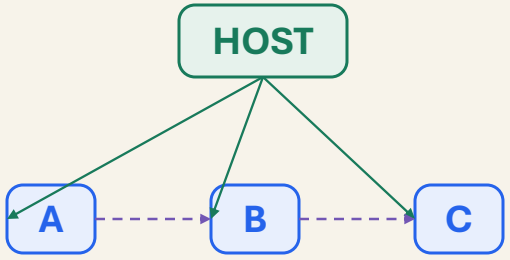
INDEPENDENT ENSEMBLE



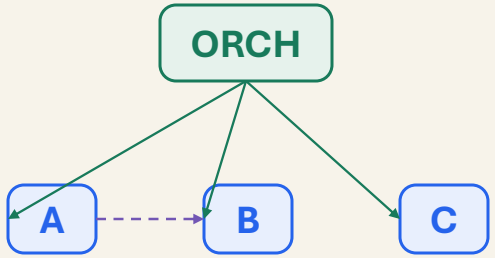
CENTRALIZED



DECENTRALIZED PEERS



HYBRID

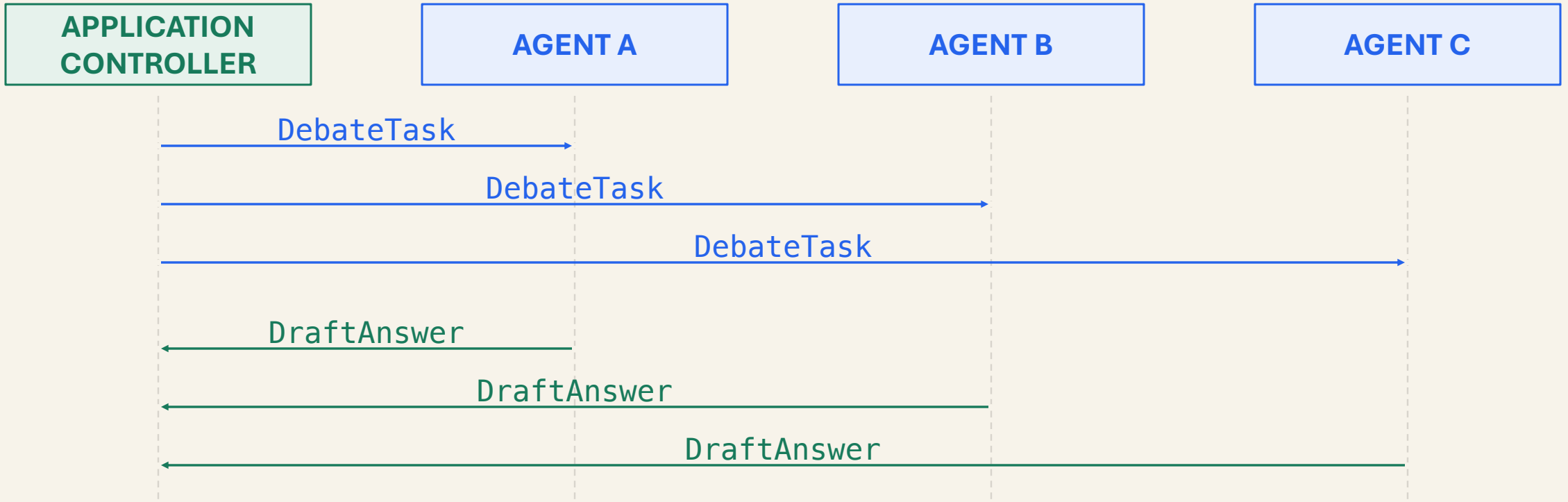


Solid = application-mediated flow. Dotted = logical peer influence.

Original course diagram · topology categories follow Kim et al. (2026)

Peer debate starts with independent first answers

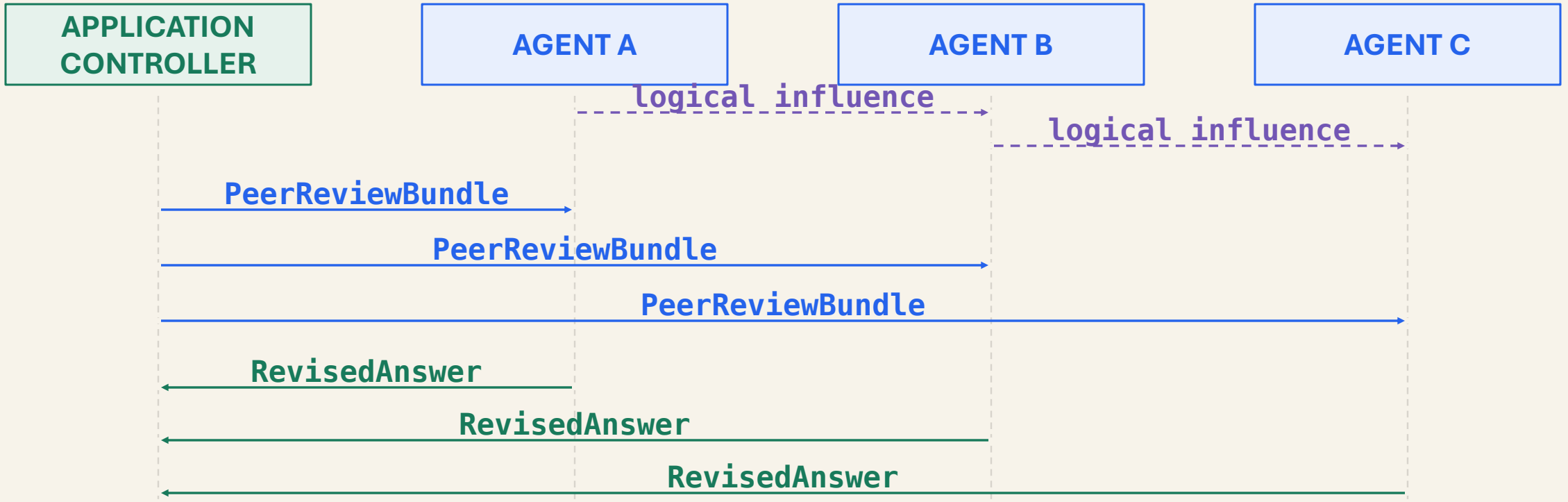
Peer debate lets agents inspect and critique one another's intermediate answers. Independent drafts come first.



- The controller sends one `DebateTask` independently to Agents A, B, and C.
- Each agent produces its own `DraftAnswer`, including an answer and supporting evidence.
- Agents cannot see each other's drafts

The application brokers bounded peer critique

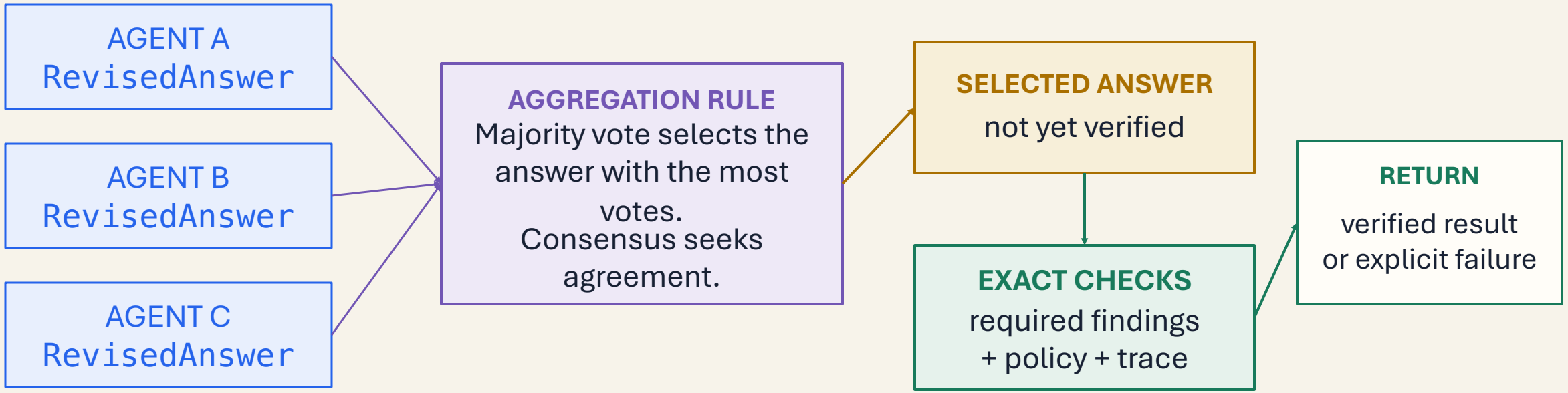
A logical peer edge means one agent's output may influence another answer. The application still transports every message.



- Agents influence one another only through controller-delivered review bundles
- A `PeerReviewBundle` is the package of material the controller gives an agent so it can reconsider its answer, which includes the agents original draft, selected drafts for critique from other agents, review criteria, etc

Aggregation selects a candidate before verification

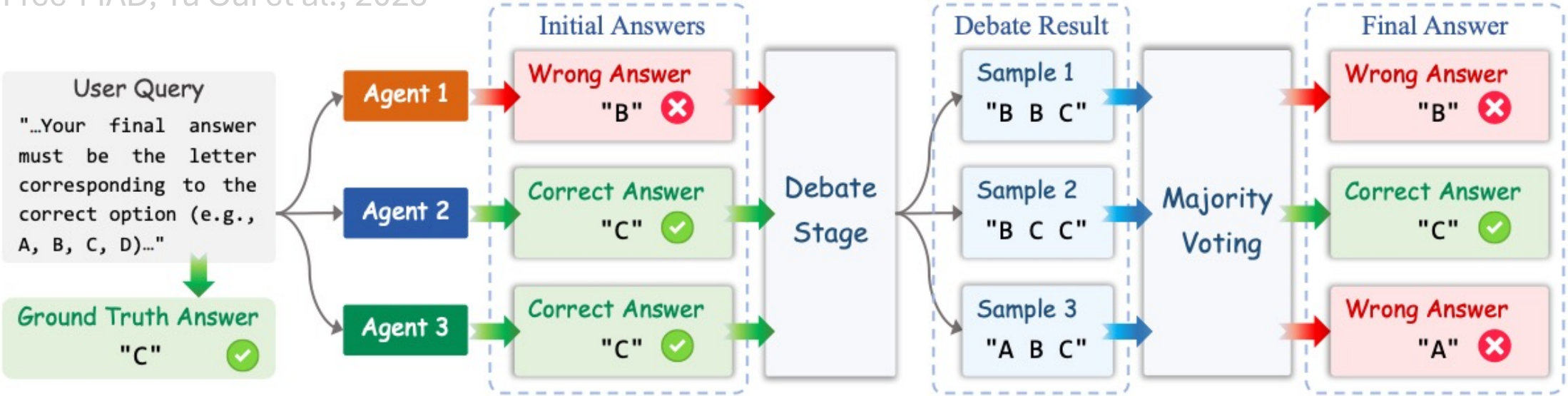
Aggregation combines normalized answers. Verification checks the selected candidate against explicit acceptance conditions.



> Agreement can select a candidate. Only application-owned verification determines whether the user may receive it.

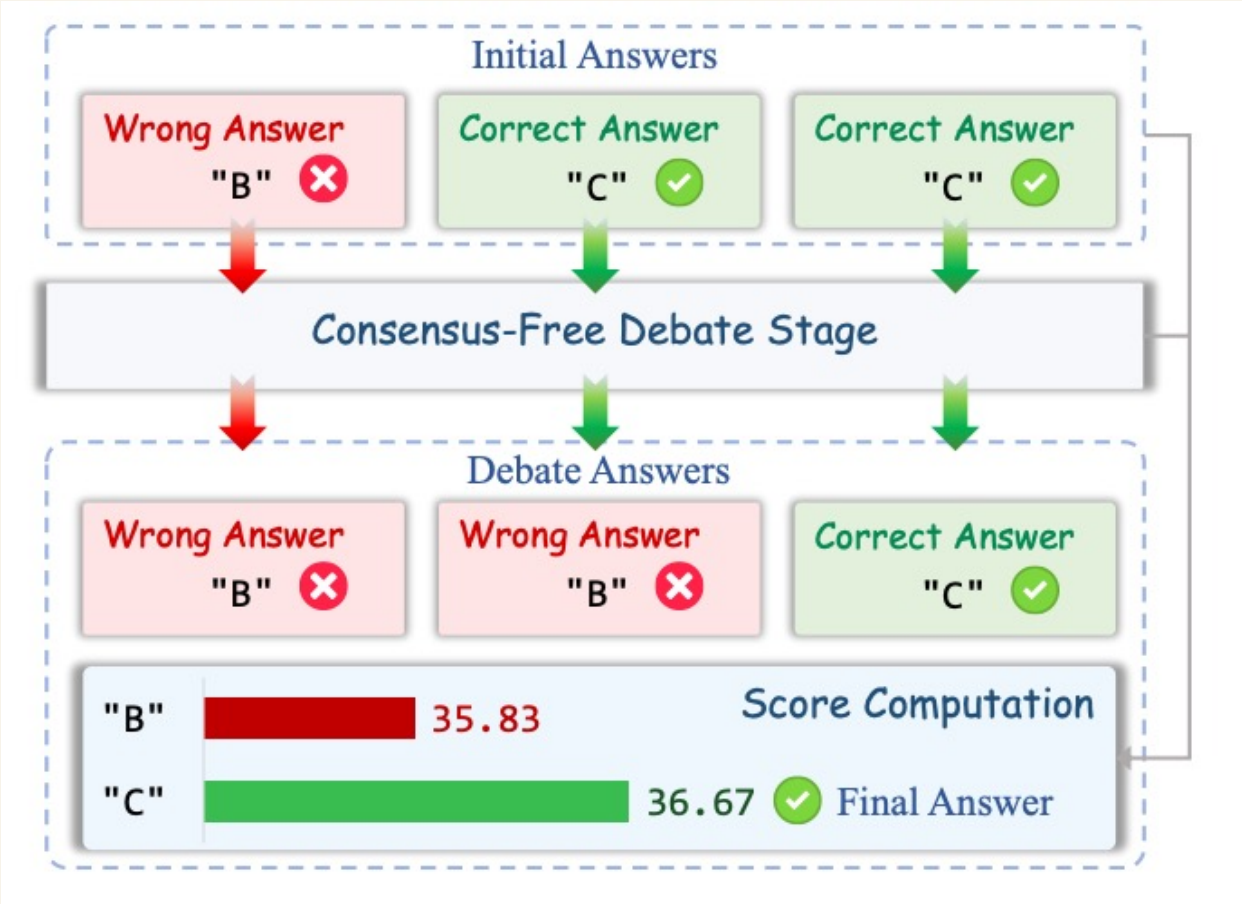
A correct minority can exist before peer influence

Free-MAD, Yu Cui et al., 2025



A multi-agent debate process showing how agents can influence each other across three independent trials (Samples 1–3). It highlights that majority voting on the post-debate answers can sometimes result in an incorrect final answer, even if the initial majority was correct.

Free-MAD scores answer trajectories without requiring consensus



HOW FREE-MAD DIFFERS

TRAJECTORY SCORE

Answers accumulate points across rounds. Initial support and staying with an answer add points; abandoning an answer removes points; adopting a new answer adds points to it. Later changes count less. The highest total wins.

ANTI-CONFORMITY PROMPTING

Agents are instructed to challenge peer reasoning and change their answer only when they find a clear error, not merely to agree with the majority.

Free-MAD changes the selection rule. When the correct answers are in the minority in the final round, the framework is still able to identify the correct one as the final answer

PAPER REVIEW

Towards a Science of Scaling Agent Systems

When does coordination improve performance, and when does it make the task harder?

Kim et al. · arXiv:2512.08296v3 · 8 April 2026

The paper asks when multi-agent coordination actually helps

Study question

*Which task **properties** and **coordination** patterns predict whether a multi-agent system will outperform one agent?*

260 experimental configurations

6 interactive benchmarks

5 agent architectures

3 model families

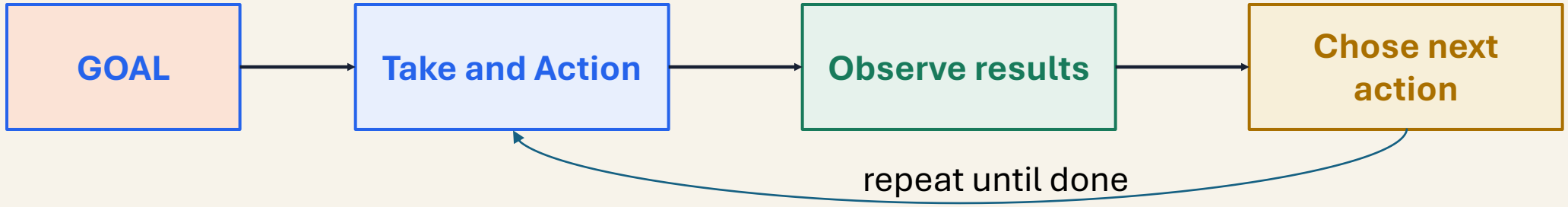
Controlled comparison

Same task interfaces	Architectures used the same benchmark tools and environment.
Matched budgets	Comparisons controlled the amount of computation available.
Task-level outcomes	Success or accuracy was measured for each benchmark.

Source: Kim et al. (2026), §§1 and 4.1.

Agentic tasks reveal the next step through interaction

The initial prompt provides the goal, but actions and tool results reveal what the system should do next.



MULTI-STEP

The result of one action determines the next

INCOMPLETE VIEW

Tools reveal information not present in the prompt

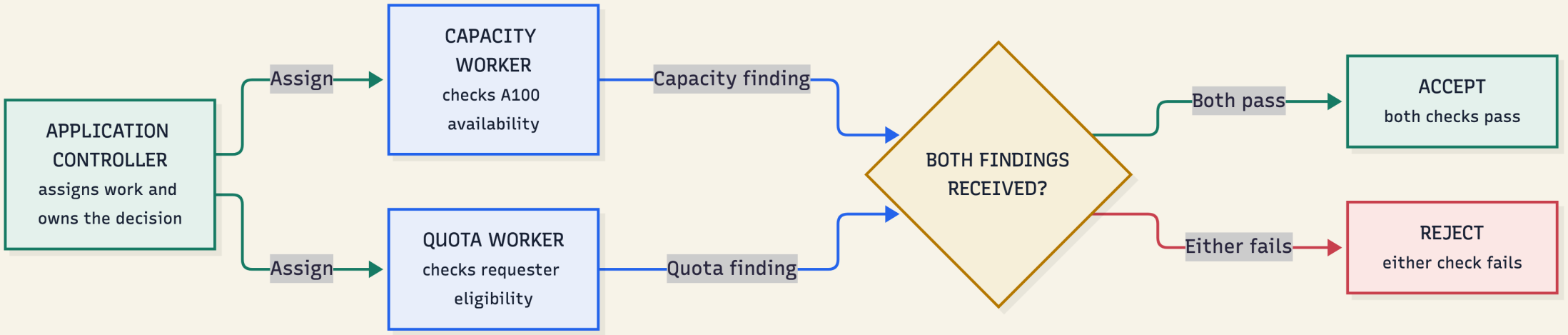
ADAPTIVE STRATEGY

The plan changes as evidence arrives.

> A task is not agentic merely because it is difficult. It is agentic when solving it requires repeated action and environmental feedback.

Source: Kim et al. (2026), §3.1.

Messages carry information; coordination decides what happens next



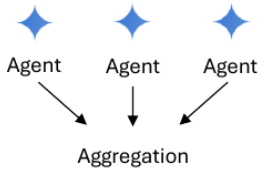
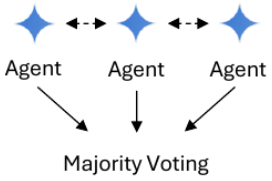
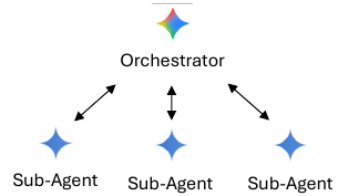
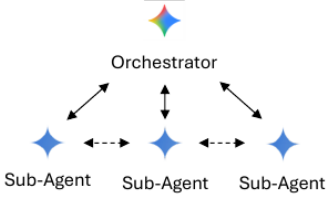
Aggregator Collects independently produced outputs. It does not direct or cross-check the work.

Orchestrator Directs sub-agents and may compare, revise, verify, or override their results.

> More messages do not guarantee better coordination. Topology is one design choice among specialization, memory, and aggregation strategy.

Source: Kim et al. (2026), §3.2.

Five architectures vary who directs the work and who can talk

Characteristic	SAS	MAS (Independent)	MAS (Decentralized)	MAS (Centralized)	MAS (Hybrid)
Interaction Type	 Agent	 Agent Agent Agent Aggregation	 Agent Agent Agent Majority Voting	 Orchestrator Sub-Agent Sub-Agent Sub-Agent	 Orchestrator Sub-Agent Sub-Agent Sub-Agent

Original interaction diagrams from the paper

SAS

One agent handles the full task.

MAS · Independent

Agents work separately. An aggregator combines outputs.

MAS · Decentralized

Peers exchange information without a central controller.

MAS · Centralized

An orchestrator assigns, compares, and verifies work.

MAS · Hybrid

Orchestrator control with limited peer exchange.

Figure excerpt: Kim et al. (2026), Table 2, p. 10.

Six benchmarks used to expose different coordination demands

BrowseComp-Plus

OPEN-WORLD RESEARCH

Navigate **websites**, find evidence, and synthesize across pages.

PlanCraft

SEQUENTIAL PLANNING

Follow **Minecraft** recipes while tracking inventory and dependencies.

SWE-bench Verified

ITERATIVE ENGINEERING

Inspect code, edit files, and run tests to resolve **GitHub issues**.

Finance Agent

DECOMPOSABLE ANALYSIS

Perform multi-step **quantitative analysis** and risk assessment.

Workbench

PROCEDURAL TOOL USE

Choose tools and carry out common **business** activities.

Terminal-Bench

CLI EXECUTION

Complete stateful **command-line tasks** across several domains.

> These are not interchangeable accuracy tests: they vary in parallelism, tool use, state, and how strongly one step constrains the next.

Source: Kim et al. (2026), §4.1.

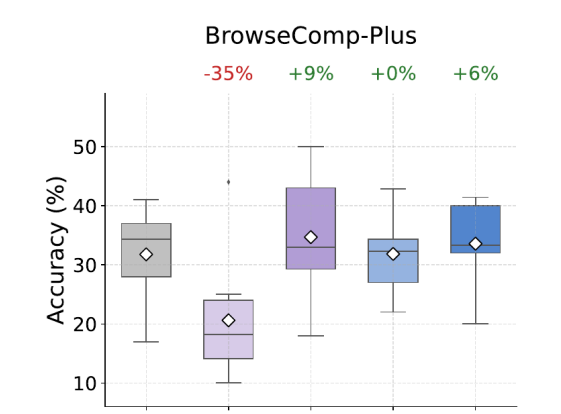
The study measures cost, traffic, overlap, efficiency, and failure

Coordination overhead	Extra **reasoning turns** used by MAS compared with its matched single-agent baseline.
Message density	How many **inter-agent messages** occur per reasoning turn.
Redundancy	How **similar the agents' outputs** are. Similarity can mean agreement or duplicated work.
Coordination efficiency	Task success normalized by the **additional turn cost** of coordination.
Trace-level error amplification	Additional token work caused when **mistakes spread** through agent interactions.

Source: Kim et al. (2026), §§3.2 and 4.4. Formulas omitted here to foreground interpretation.

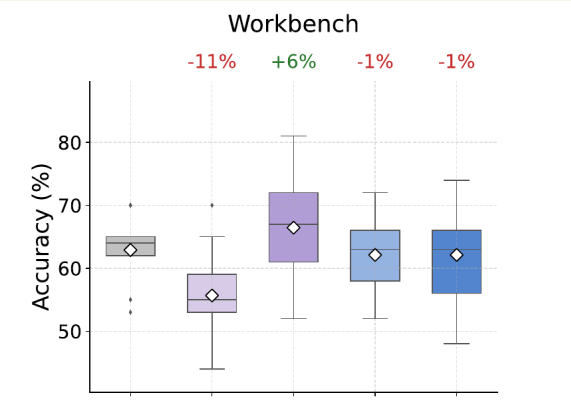
Across four benchmarks, coordination gains were small or inconsistent

Each colored box summarizes several results obtained by testing the same architecture with different models/configurations. Higher box = generally better accuracy.



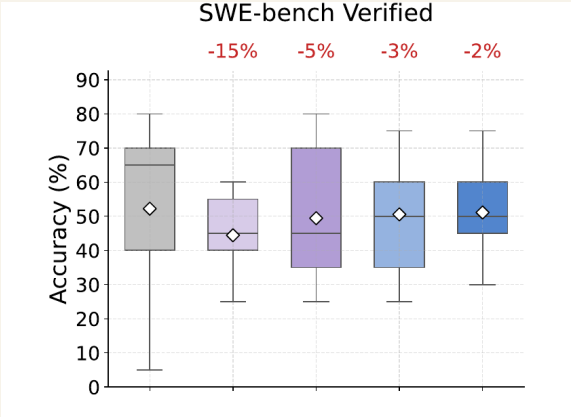
MULTI-SITE WEB RESEARCH

Independent MAS fell 35%. The other three variants ranged from 0% to +9%.



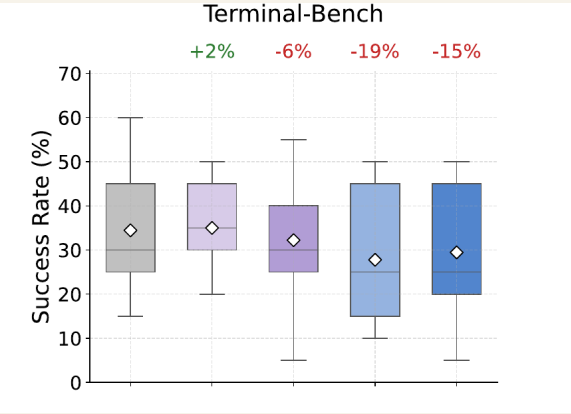
TOOL-BASED BUSINESS TASKS

Every change was modest, ranging from −11% to +6% relative to SAS.



REAL GITHUB ISSUE RESOLUTION

Every MAS variant was worse than SAS, with relative declines from 2% to 15%.



STATEFUL COMMAND-LINE TASKS

Independent MAS gained 2%. The other variants declined by 6% to 19%.

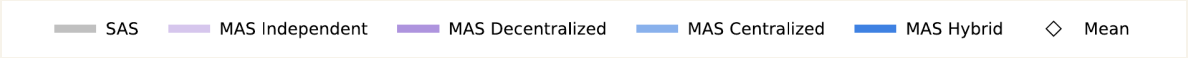


Figure excerpts: Kim et al. (2026), Fig. 2, p. 11. Percentages are relative changes, not percentage points.

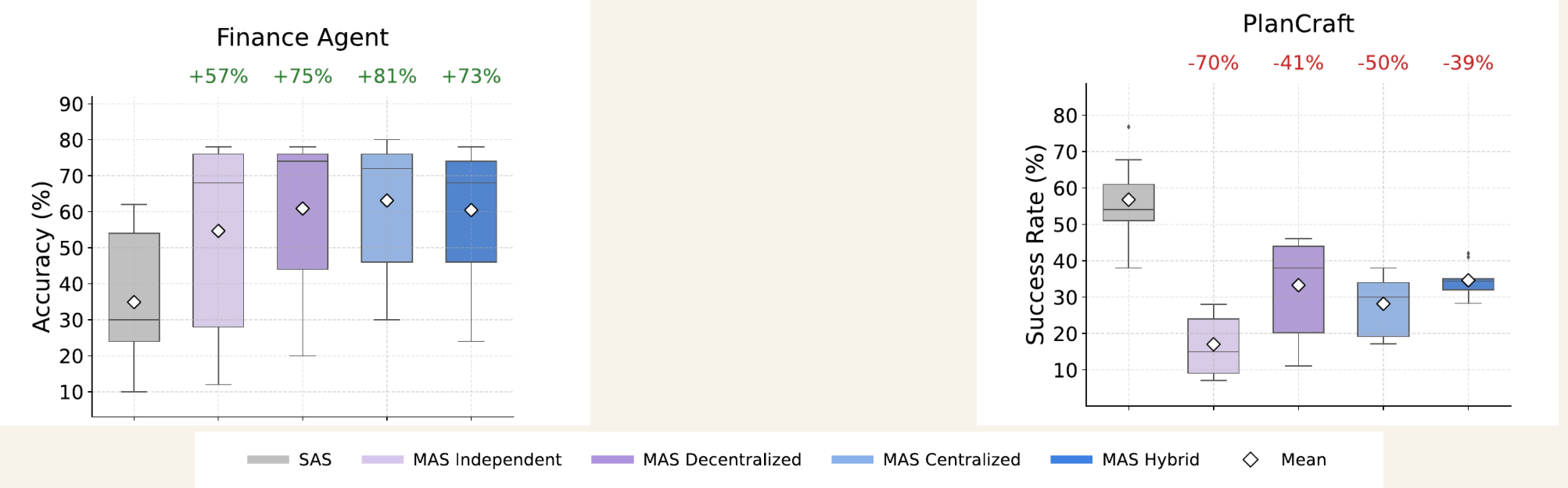
Parallel financial analysis improved while sequential crafting degraded

FINANCE AGENT · multi-step financial analysis

Research subtasks can proceed independently before a central orchestrator combines the findings.

PLANCRAFT · sequential Minecraft crafting

The agent follows recipes, tracks inventory, and completes dependent actions in the correct order.



Centralized mean 0.631 vs SAS 0.349 · +80.8%

Best MAS: Hybrid 0.346 vs SAS 0.568 · −39.1%

Figure excerpts: Kim et al. (2026), Fig. 2, p. 11. Percentages are relative changes, not percentage points.

Three patterns characterize when coordination helps, or hurts

1 CAPABILITY SATURATION

Once the single-agent baseline was already strong, roughly above **45%, additional coordination produced diminishing or negative returns**.

Strong baseline, smaller gain

2 TOOL USE AND COORDINATION EFFICIENCY

Under a matched total token budget, tool-heavy tasks left less capacity for reasoning and coordination, reducing multi-agent efficiency.

More tool use, lower efficiency

3 Coordination structure affects how far errors propagate

In this study, compared with the single-agent baseline:

- Independent agents showed **17.2× more error-related work**.
- Centrally coordinated agents showed **4.4× more**.

> Agent count alone was not a significant performance predictor. Architecture and task alignment explained more than team size.

Adapted from Kim et al. (2026), Tables 4–5 and §5.2.

SECTION 8

Synthesis, implementation, and exit check

Can you justify both the control method and any multi-agent coordination operation?

TECHNICAL MAPPING

Success follows completed procedure and verification—not a confident proposal.



NASA, Apollo 13 Mission Control after splashdown

Choose a control method and coordination operation separately

1 · CONTROL METHOD

Fixed workflow
Use when the sequence is known.

Step-by-step agent loop
Use when the next action depends on the latest observation.

Planner-based workflow
Use when explicit dependencies and parallel work justify a plan.

2 · COORDINATION OPERATION, IF NEEDED

Routing
Select the initial handler.

Delegation
Call another agent for a subtask and receive a result.

Handoff
Transfer the active interaction and later responses.

> Agent count is a separate topology choice; it is not a fourth control method.

Architecture worksheet: assumptions and a disconfirming test

State the task property that motivates the design, then name evidence that would show the design is wrong.

01 · TASK + SUCCESS
Recommend one A100 only if all six acceptance criteria pass.

02 · CONTROL METHOD
Fixed workflow: required evidence and order are known.

03 · COORDINATION
Delegation is optional for capacity and quota subtasks.

04 · SIDE-EFFECT POLICY
`forbiddenActions` = ["`provisionGpu`"]

05 · FAILURE POLICY
A missing required finding produces `ExplicitFailure`.

06 · DISCONFIRMING MEASUREMENT
More failures or latency with no accuracy or auditability gain.

> A design claim should name both its motivating assumption and a measurement that could reject it.

Exit check: choose both design axes

SCENARIO

A support request may need billing, technical, or account expertise. After every specialist result, the next useful action depends on what was learned. One agent should continue the user-facing interaction.

ANSWER 1 · CONTROL METHOD

Choose fixed workflow, step-by-step agent loop, or planner-based workflow. Name the task property that supports your choice.

ANSWER 2 · COORDINATION OPERATION

Choose routing, delegation, handoff, or none. State which component remains responsible for the next user-facing response.

> Do not answer with agent count alone; explain control and responsibility separately.

Exit-check debrief: one defensible design

CONTROL METHOD

Step-by-step agent loop (ReAct-style)

Why: the next useful action depends on the latest specialist or tool result, so a fixed sequence may perform unnecessary work.

COORDINATION OPERATION

Delegation for narrow specialist calls

Why: one user-facing agent should retain responsibility, request a specialist result, and then continue the interaction.

PLAUSIBLE ALTERNATIVE

Use handoff when the specialist should own later responses; use a planner when explicit dependencies justify a plan.

> The correct design depends on task properties and responsibility—not on which label sounds most agentic.

More agents help when the task can be divided or checked independently

When useful work can proceed in parallel, coordination can improve the result.

When every step changes the state for the next step, extra agents can add delay and errors.

Choose the coordination pattern after you understand the task.

Some Reference Papers

Each source supports a different part of the design argument.

Du et al. (2023) · arXiv:2305.14325

Introduces multi-agent debate with proposal, peer inspection, and revision across rounds.

Oh et al. (2026) · TACL 14

Diagnoses belief entrenchment and majority amplification in homogeneous debate.

Choi, Zhu, and Li (2025) · arXiv:2508.17536

Separates independent voting from debate and tests both across seven NLP benchmarks.

Cui et al. (2026) · ACL Findings 1600

Proposes Free-MAD with trajectory scoring and anti-conformity prompting.

Kaesberg et al. (2025) · ACL Findings 606

Compares seven decision protocols and measures task-specific effects of voting and consensus.

Kim et al. (2026) · arXiv:2512.08296v3

Compares five architectures and connects gains or losses to task structure and coordination overhead.

Mechanism · baseline · decision protocol · failure mode · alternative rule · system architecture